

```
Enter coordinates of point 1 (x1, y1): 0 0
Enter coordinates of point 2 (x2, y2): 4 0
Enter coordinates of point 3 (x3, y3): 0 3
```

Coordinates of the points:

Point 1: (0, 0)

Point 2: (4, 0)

Point 3: (0, 3)

Area of the triangle: 6.00

The points form a valid triangle.

Array of random numbers using while loop:

```
64 20 22 75 94 83 26 94 1 53 98 85 77 62 99 86 72 68 19 61 49 47 69 75 55 40
 98 38 48 66 75 63 85 96 89 79 30 15 72 31 19 21 67 95 82 65 32 54 84 2
14 33 48 35 7 2 26 57 91 25 22 66 87 6 61 76 36 43 90 59 25 60 80 91 6
13 7 37 66 91 38 32 75 37 66 81 38 91 37 28 15 10 45 53 16 6 28 3 100 69
```

Array of random numbers using do-while loop:

```
54 84 42 61 54 45 96 18 29 29 74 82 39 33 10 70 85 86 33 66 36 4 17 74 7 94
  9 94 5 100 44 10 83 85 71 36 81 18 5 9 98 31 90 89 63 99 58 100 85 91 17
  72 46 33 98 5 78 6 98 34 5 93 44 40 77 66 27 57 83 32 65 81 62 7 21 76 5
  78 27 41 20 43 13 66 27 62 70 5 19 19 38 24 11 33 63 39 98 41 47 33
```

=== Code Execution Successful ===

Array of random numbers using for loop:

```
86 52 94 7 44 91 96 41 19 67 93 97 76 20 77 46 26 67 83 52 41 41 60 29 62 9
  92 61 2 15 74 87 67 19 45 10 10 41 50 80 59 94 77 34 13 53 32 39 72 14
  42 64 55 54 92 68 14 36 28 16 50 53 54 68 24 51 78 33 91 79 12 1 25 40
  34 37 45 17 27 16 83 21 79 89 74 22 56 39 9 35 54 59 39 60 26 62 10 55
  46 52
```

=== Code Execution Successful ===

For int data type:

Largest value: 2147483647

Smallest value: -2147483648

For long data type:

Largest value: 9223372036854775807

Smallest value: -9223372036854775808

For float data type:

Largest value: 3.402823e+38

Smallest value (positive): 1.175494e-38

For double data type:

Largest value: 1.797693e+308

Smallest value (positive): 2.225074e-308

Adding 1 to the largest value of an int (2147483647):

Result: 2147483648

=== Code Execution Successful ===

The generated 100 random real numbers:

18.70	12.63	10.97	10.45	14.52	11.06	18.43	15.61	10.55	18.97
19.41	11.45	18.57	18.25	13.21	10.38	18.06	12.91	14.52	11.34
15.98	18.17	12.81	19.38	19.01	18.17	19.44	18.39	18.55	17.44
13.69	10.77	10.08	14.67	11.22	18.12	15.73	13.17	13.73	19.81
15.66	16.66	14.78	14.24	14.92	11.51	18.14	12.98	14.42	12.66
17.84	13.92	10.84	14.17	13.30	13.37	12.35	16.27	11.76	14.42
13.71	15.46	15.19	17.31	13.65	16.41	15.44	12.90	19.58	12.69
16.23	15.24	19.36	11.02	13.00	14.28	12.53	11.15	10.78	10.48
13.81	12.15	14.40	18.17	16.32	17.71	11.54	12.19	13.98	16.83
16.61	11.21	15.81	11.80	18.53	19.46	11.73	13.97	15.88	14.83

The 100 random real numbers in descending order:

19.81	19.58	19.46	19.44	19.41	19.38	19.36	19.01	18.97	18.70
18.57	18.55	18.53	18.43	18.39	18.25	18.17	18.17	18.17	18.14
18.12	18.06	17.84	17.71	17.44	17.31	16.83	16.66	16.61	16.41
16.32	16.27	16.23	15.98	15.88	15.81	15.73	15.66	15.61	15.46
15.44	15.24	15.19	14.92	14.83	14.78	14.67	14.52	14.52	14.42
14.42	14.40	14.28	14.24	14.17	13.98	13.97	13.92	13.81	13.73
13.71	13.69	13.65	13.37	13.30	13.21	13.17	13.00	12.98	12.91
12.90	12.81	12.69	12.66	12.63	12.53	12.35	12.19	12.15	11.80
11.76	11.73	11.54	11.51	11.45	11.34	11.22	11.21	11.15	11.06
11.02	10.97	10.84	10.78	10.77	10.55	10.48	10.45	10.38	10.08

Enter the order of the matrix (rows columns):

3 3

Enter the matrix elements row-wise:

9 8 7

6 5 4

3 2 1

The given matrix is:

9 8 7

6 5 4

3 2 1

The transposed matrix is:

9 6 3

8 5 2

7 4 1

=== Code Execution Successful ===

Enter the 9 elements of matrix:

1 2 3

0 1 4

5 6 0

The matrix is

1 2 3

0 1 4

5 6 0

Inverse of matrix is:

-24.00 20.00 -5.00

18.00 -15.00 4.00

5.00 -4.00 1.00

=== Code Execution Successful ===

```
Enter first string:
SAI
Enter second string:
JYOTHI
Enter the upper and lower limits of the substring:
2 4
Length of first string: 3
Copied string: SAI
Reversed string: IAS
Concatenated string: SAIJYOTHI
Substring from 2 to 4: AIJ
```

```
=== Code Execution Successful ===
```

```
Enter the value of num:
6
Enter the elements one by one:
12
18
4
24
25
42
Sorted array is:
4
12
18
24
25
42
Enter the element to be searched:
4
SEARCH ELEMENT FOUND
```

```
=== Code Execution Successful ===
```

STACK OPERATION

- 1. PUSH
- 2. POP
- 3 DISPLAY
- 4 EXIT

Enter your choice

1

Enter the element to be pushed

3

Do you want to continue(Type 0 or 1)?

1

- 1. PUSH
- 2. POP
- 3 DISPLAY
- 4 EXIT

Enter your choice

1

Enter the element to be pushed

4

Do you want to continue(Type 0 or 1)?

1

- 1. PUSH
- 2. POP
- 3 DISPLAY
- 4 EXIT

Enter your choice

3

The status of the stack is

4

3

Do you want to continue(Type 0 or 1)?

1

- 1. PUSH
- 2. POP
- 3 DISPLAY
- 4 EXIT

Enter your choice

2

poped element is = 4nDo you want to continue(Type 0 or 1)?

0

Queue is empty!!

Inserted -> 1

Inserted -> 2

Inserted -> 3

Inserted -> 4

Inserted -> 5

Queue is full!!

Front -> 0

Items -> 1 2 3 4 5

Rear -> 4

Deleted element -> 1

Front -> 1

Items -> 2 3 4 5

Rear -> 4

Inserted -> 7

Front -> 1

Items -> 2 3 4 5 7

Rear -> 0

Queue is full!!

```
Enter the first polynomial:

Enter the coefficient as -999 to end the polynomial

Enter term 1:
Coeff = 3
Pow x = 4

Enter term 2:
Coeff = 5
Pow x = 2

Enter term 3:
Coeff = 6
Pow x = 0

Enter term 4:
Coeff = -999

Enter the second polynomial:

Enter the coefficient as -999 to end the polynomial

Enter term 1:
Coeff = 4
Pow x = 3

Enter term 2:
Coeff = 1
Pow x = 2

Enter term 3:
Coeff = 6
Pow x = 0

Enter term 4:
Coeff = -999

Polynomial 1: 3x^4 + 5x^2 + 6x^0
Polynomial 2: 4x^3 + 1x^2 + 6x^0
Polynomial Result: 3x^4 + 4x^3 + 6x^2 + 12x^0
```

```
=== Code Execution Successful ===
```

```
How many elements are u going to enter?: 8  
Enter 8 elements: 76 34 66 23 11 654 22 89  
Order of Sorted elements: 11 22 23 34 66 76 89 654
```

```
=== Code Execution Successful ===
```

```
Enter the size: 7
Enter the elements of the array: 34 65
34 65 32 78 54 21 59
The sorted array is: 32 34 34 54 65 65 78
```

```
=== Code Execution Successful ===
```

Inorder traversal:

5 -> 12 -> 6 -> 1 -> 9 ->

Preorder traversal:

1 -> 12 -> 5 -> 6 -> 9 ->

Postorder traversal:

5 -> 6 -> 12 -> 9 -> 1 ->

=== Code Execution Successful ===

```
Depth First Search: S A D B C
```

```
=== Code Execution Successful ===
```